Module Code & Module Title
CS6004NT- Artificial Intelligence

Assessment Weightage & Type

Individual Coursework (75%)

Year and Semester

2024-25 Autumn

Student Name: Anir Jung Thapa

London Met ID: 22072054

College ID: np05cp4a220015@iic.edu.np

Assignment Submission Date: 22nd January 2025

Submitted To: Mr. Zishan Siddique

I confirm that I understand my coursework needs to be submitted online via Google Classroom under the relevant module page before the deadline for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

Table of Contents

1 Introduction 1
1.1 Explanation of the AI topic/concepts used 2
1.2 Explanation/introduction of the chosen problem domain/topic 3
2 Background 5
2.1 Research work done on the chosen topic/problem domain 5
2.2 Research Work on Chosen Topic/Problem Domain 6
2.3 Review and Analysis of the Existing Works 7
2.3.1 Research 1: Machine Learning Techniques for Sentiment Analysis: A Review 8
2.3.2 Research 2: A Survey on Sentiment Analysis Methods, Applications, and Challenges 10
2.4 Existing Challenges 12
2.5 Dataset 13
2.6 Dataset Information 13
2.7 Dataset Background 13
3 Solution 15
3.1 Explanation of purposed solution 15
3.2 Explanation of the algorithms used 16
3.2.1 Long short-term memory (LSTM) 16
3.2.2 Logistic Regression 20
3.2.3 Naive Bayes 23
3.3 Diagrammatical representations of the solution 27
3.4 Development process requirements 30
3.4.1 Libraries used: 30
3.5 Explanation of the development process 33
3.5.1 Importing libraries 33
3.5.2 Setting up the data that will be used later for training the ai model 34
3.5.3 Preprocessing data 35
3.5.4 Looking for possible imbalance in dataset labels 36
3.5.5 Cleaning Dataset 38

3.5.6 Split the Data 39
3.5.7 Select Model and training Model 40
3.5.8 Comparison 47
4 Conclusion 49
4.1 Analysis of work done 49
4.2 How the solution addresses the real-world problems 50
4.3 Further work 51
5 References 53
6 Originality Report 55

Table of Figures

Figure 1: report by International Journal of Multidisciplinary sciences and Engineering . 8
Figure 2: research on sentiment analysis by Mayur Wankhede and this team..... 10
Figure 3: Architecture of an LSTM 16
Figure 4: Architecture of a Logistic Regression Model 20
Figure 5: how naive bayes algorithm works 23
Figure 6: flowchart of LSTM model..... 27
Figure 7: Flowchart of Logistic Regression 28
Figure 8: Flowchart of Naive Bayes..... 29
Figure 9: pandas logo..... 30
Figure 10: NumPy logo..... 31
Figure 11: Matplotlib logo 31
Figure 12: Scikit Learn logo..... 32
Figure 13: tensor flow logo 32
Figure 14: import statements..... 33
Figure 15: loading data for from dataset and checking data frame 34
Figure 16: looking of imbalance dataset..... 35
Figure 17: graph showing imbalance in dataset 36
Figure 18: over sampling dataset to make labels balanced..... 37
Figure 19: preprocessing data..... 38
Figure 20: graph showing balanced label after over sampling 38
Figure 21: splitting datasets for training and testing 39
Figure 22: tokenizing text to train in LSTM model 39
Figure 23: vectorizing text to train in Naive bayes and Logistic Regression..... 39
Figure 24: Creating parameters for LSTM model 40
Figure 25: creating model and training LSTM model..... 41
Figure 26: setting model for Logistic regression and naive bayes..... 42
Figure 27: training Naive bayes and Logistic regression model 42
Figure 28: Evaluating Naive Bayes and Logistic Regression 43
Figure 29: evaluating LSTM model..... 45
Figure 30: website developed to predict sentiments using best model that was developed 48
Figure 31: Predicting sentiments using developed website..... 48
Figure 32: Predicting Sentiments 48

Table of Tables

Table 1: sample dataset for example of LSTM 17
Table 2: sample dataset for example of Logistic Regression 21
Table 3: sample dataset for example of Naive Bayes 24
Table 4: Summary table of work analysis 50

1 Introduction

Artificial Intelligence (AI) is a branch of computer science dedicated to creating machines capable of performing tasks that typically require human intelligence. These include thinking, decision-making, problem-solving, understanding language, and recognizing speech, all autonomously. AI integrates deeply into our daily lives, powering virtual assistants, chatbots, recommendation systems, and autonomous vehicles. It learns from data, adapts to new challenges, and processes complex information, revolutionizing industries by enhancing efficiency and creating opportunities in healthcare, education, and entertainment. (Glover, 2024)

The development of AI has spanned decades, beginning with foundational theories in the 1950s and progressing despite early challenges, including periods of stagnation known as "AI Winters." Recently, AI has seen a resurgence, fueled by advances in GPU technology for faster processing and the availability of extensive data sets. These advancements, along with breakthroughs in machine learning and deep learning, have positioned AI as a driving force in modern technological innovation (Coursera, 2024) (rapidqube, 2024).

AI can be categorized into different types based on functionality:

- **Artificial Narrow Intelligence (ANI):** Today's AI, which excels in specific tasks such as powering virtual assistants like Siri and Alexa or platforms like ChatGPT.
- **General Intelligence (GI):** A theoretical form of AI that could perform any intellectual task with human-like capability without specific training.
- **Super Intelligence (SI):** Also theoretical, this form of AI would surpass human intellect in all aspects, potentially possessing its own emotions and desires (IBM Data and AI Team, 2023).

1.1 Explanation of the AI topic/concepts used

Sentiment analysis is an automated technique that identifies and categorizes emotions expressed in text, determining if sentiments are positive, negative, or neutral. This method is essential for analyzing text from social media, customer reviews, and other sources to assess public opinions on various products, services, or topics. It plays a crucial role in helping businesses monitor their brand image, comprehend customer preferences, and make informed decisions by analyzing the emotional undertones in the feedback they receive.

To conduct sentiment analysis, this project utilizes **Natural Language Processing (NLP)**, a subset of artificial intelligence that enables computers to understand and interact with human language. NLP works by simplifying complex language into more manageable parts, Understanding the meanings of these parts, and facilitating interactions that humans can easily understand. This technology is crucial for tasks such as language translation, customer service automation, content summarization, and sentiment detection. Its ability to process large volumes of text efficiently makes it invaluable for managing extensive data from digital platforms, enhancing the effectiveness of chatbots, and supporting other language-based applications.

To enhance our sentiment analysis capabilities, this project utilizes **Long Short-Term Memory (LSTM)** networks, a sophisticated type of **recurrent neural network (RNN)** that is particularly well-suited for handling tasks involving sequences of data. Unlike traditional RNNs, which can struggle with longer sequences due to context loss or the vanishing gradient problem, LSTMs excel at maintaining context over extended periods. This makes them ideal for analyzing the flow of language in longer texts, where understanding the progression and subtleties of sentiment is crucial. By implementing LSTM networks, we can significantly enhance the precision and reliability of our sentiment analysis, effectively capturing intricate sentiment dynamics during extended dialogues or complex storylines.

1.2 Explanation/introduction of the chosen problem domain/topic

The primary focus of this research project is the significant impact of social media on mental health, particularly its role in exacerbating conditions such as anxiety, depression, and low self-esteem, which constitutes the core problem domain. Social media has become a double-edged sword; while it connects us, it also serves as a stage for expressing negative emotions, which can profoundly affect others. This dynamic can lead to a cycle of negativity, amplifying feelings of anxiety, depression, and low self-esteem among its users. Indeed, studies reinforce this link, such as research published in the "**Journal of Clinical Psychology**" that associates heavy social media use with increased anxiety and depression, particularly among teenagers (Berger, 2018). According to the **American Psychological Association (APA),** adults who spend more than six hours a day on social media report higher stress levels compared to those who use it less frequently. Additionally, exposure to negative content on these platforms can lead to 'emotional contagion,' spreading distress among users (DeAngelis, 2020).

The project employs three distinct machine learning models Long Short-Term Memory (LSTM) networks, Naïve Bayes, and Logistic Regression independently to conduct sentiment analysis on social media posts. This multi-model approach enables a comprehensive analysis of textual material, where each model contributes uniquely:

- **LSTM Networks**: These are adept at processing sequences and can capture the temporal evolution of language used in social media, making them particularly useful for detecting subtle shifts in emotional states that may indicate mental health issues.
- **Naïve Bayes**: This model is excellent for baseline sentiment classification due to its simplicity and efficiency in handling large datasets with high-dimensional input.
- **Logistic Regression**: Often used for its interpretability, Logistic Regression provides a reliable linear model for determining the impact of individual words or features on the sentiments expressed.

By analyzing patterns and variations in the data, the project aims to identify emotional trends and potential mental health signals, each model enhancing the robustness and reliability of the analysis. This methodical application of different analytical techniques

ensures a broad and nuanced perspective on the sentiments expressed across social media platforms.

2 Background

Sentiment analysis is an important part of natural language processing, helping us extract and understand public opinions from various text sources like reviews, comments, and social media posts. Its applications are wide-ranging, from business intelligence and customer satisfaction studies to predicting trends, making it a valuable tool in today's digital world. However, the effectiveness of sentiment analysis depends heavily on the algorithms used, as each has its own strengths and weaknesses. This research aims to compare the performance of three different machine learning algorithms on a specific dataset to determine the best approach for accurate and efficient sentiment analysis. By doing so, it seeks to fill existing knowledge gaps and improve the reliability of sentiment predictions.

2.1 Research work done on the chosen topic/problem domain

The impact of social media on mental health has been a prominent subject of research, with numerous studies highlighting its negative consequences. Social media, while providing a platform for communication and connection, has also been linked to increasing rates of anxiety, depression, and low self-esteem. Researchers have identified various contributing factors, such as excessive screen time, exposure to cyberbullying, and the constant pressure to present an idealized version of oneself online.

A study by the "**Royal Society for Public Health**" found that prolonged use of platforms like Instagram and Snapchat is associated with increased feelings of inadequacy and anxiety, particularly among teenagers. The study also reported that 70% of young people felt that social media negatively impacts their mental health (RSPH, 2023). Additionally, the American Psychological Association reported that individuals who spend more than six hours per day on social media are significantly more likely to experience heightened stress levels, often driven by exposure to negative content or comparison with others (DeAngelis, 2020).

Cyberbullying, another byproduct of social media, has emerged as a significant factor contributing to mental health problems. Research conducted by the Pew Research Center in 2021 found that 59% of U.S. teens have been victims of online harassment.

This form of bullying often leaves victims feeling isolated and distressed, compounding the mental health challenges they face (Kaspersky Lab, 2024).

Furthermore, social media can amplify issues such as "fear of missing out" (FOMO), where users feel excluded or left behind when viewing curated posts of others' lives. Studies have shown that this phenomenon can lead to feelings of inadequacy and loneliness, particularly among young adults and adolescents (Bergman, 2024).

These findings underline the urgent need to address the psychological challenges posed by social media usage. Research continues to explore how these platforms influence users' emotional well-being and what steps can be taken to mitigate their adverse effects. By better understanding these impacts, we can develop strategies to promote healthier online interactions and improve mental health outcomes (Naslund, 2012).

2.2 Research Work on Chosen Topic/Problem Domain

Sentiment analysis is increasingly recognized for its ability to gauge emotional trends and mental health indicators from social media content. This approach, grounded in Natural Language Processing (NLP), uses advanced computational methods to interpret and categorize emotions expressed in text. Recent technological advancements have significantly improved our capability to analyze large volumes of data from platforms like Twitter, Facebook, and Instagram, making it possible to detect subtle shifts in mood and sentiment (AWS, 2024).

For example, research published in the "**Journal of Clinical Psychology**" highlights a correlation between extensive social media use and increased levels of anxiety and depression, particularly among teenagers. These studies show that teenagers spending several hours per day on social media platforms are more likely to report mental health issues. This insight is crucial for developing tools that can monitor and react to signs of mental distress (Berger, 2018).

Our project utilizes Long Short-Term Memory (LSTM) networks, a type of recurrent neural network, to delve into the nuances of how sentiments on social media evolve over time. By tracking these changes, LSTMs can identify emerging patterns indicative of mental health concerns. For instance, if a user's posts gradually become more negative or anxious over several weeks, this could signal a worsening mental state.

Current research, however, often overlooks long-term effects and the impact of different types of social media interactions. Additionally, most tools are not equipped to provide immediate interventions or support in response to detected emotional changes.

To address these gaps, future studies might integrate systems that offer real-time support or interventions directly through social media platforms. Imagine a scenario where a system detects a troubling pattern in a user’s posts and automatically sends a supportive message or mental health resources tailored to their apparent needs.

By further developing sentiment analysis techniques and applying them to the vast data generated on social media, we can enhance our understanding of its impact on mental health. This work not only contributes to academic research but also provides practical tools for early detection and intervention, potentially mitigating the negative psychological effects of social media usage.

2.3 Review and Analysis of the Existing Works

Many studies have investigated the progress and challenges of sentiment analysis, showing its growing importance in different industries. Researchers have tested methods like Naive Bayes, Support Vector Machines, LSTM (Long Short-Term Memory), and Deep Learning to improve how accurately sentiments are classified. They’ve also worked on better ways to clean data, handle unbalanced datasets, and apply sentiment analysis to specific areas. These studies give useful insights into what works well and what does not, creating a strong starting point for further research and comparison.

2.3.1 Research 1: Machine Learning Techniques for Sentiment Analysis: A Review

INTERNATIONAL JOURNAL OF MULTIDISCIPLINARY SCIENCES AND ENGINEERING, VOL. 8, NO. 3, APRIL 2017

Machine Learning Techniques for Sentiment Analysis: A Review

Munir Ahmad[1], Shabib Aftab[2], Syed Shah Muhammad[3] and Sarfraz Ahmad[4]
[1-4]Department of Computer Science, Virtual University of Pakistan
[1]munirahmad@gmail.com, [2]shabib.aftab@gmail.com

***Abstract*– Social media platforms and micro blogging websites are the rich sources of user generated data. Through these resources, users from all over the world express and share their opinions about a variety of subjects. The analysis of such a huge amount of user generated data manually is impossible, therefore an effective and intelligent technique is needed which can analyze and provide the polarity of this textual data. Multiple tools and techniques are available today for automatic sentiment classification for this user generated data. Mostly, three approaches are used for this purpose Lexicon based techniques, Machine Learning based techniques and hybrid techniques (which combines lexicon based and machine learning based approach). Machine Learning approach is effective and reliable for opinion mining and sentiment classification. Many variants and extensions of machine learning techniques and tools are available today. The purpose of this study is to explore the different machine learning techniques to identify its importance as well as to raise an interest for this research area.**

***Keywords*– Machine learning, Sentiment Analysis, Opinion mining, Social media, Polarity Detection**

I. INTRODUCTION

Due to the rapid growth of social media, bulk of user generated data is available now. Analyzing the sentiments and accurate classification of this gigantic amount of data is a very challenging task. Most of the data available on the internet is in the textual form as it is the most natural and readable form for presenting the thoughts and opinions to

These algorithms classify the input data set with the help of trained classifier. Training data is composed of a set of training examples, each of them comprise of input object and desired output results [4]. An inferred function is created by analyzing the training data by supervised learning methods that can be later used for mapping new incoming data which is also called the test data. Mostly machine learning techniques use the Supervised approach. It can be further categorized in two methodologies i.e., Classification and Regression. Most common examples of supervised machine learning algorithms are Linear Regression, Random Forest and Support Vector Machines.

Un-supervised: These type of machine learning algorithms takes the unlabeled input data and then with the help of different algorithms hidden structure/pattern is discovered Unlike the supervised learning this technique does not use the pre labeled data to train the classifier. Un-supervised machine learning can be further divided into clustering and association, the most common example of Un-supervised machine learning algorithms are K-Means and Apriori Algorithm.

Semi-Supervised: These type of algorithms deal with the both labeled and unlabeled data sets. [6] reviewed different lexicon based tools and techniques and mentioned the comparison between the features and accuracy results of different lexicon techniques. Taking it a step ahead, different Machine Learning techniques/algorithms are studied and analyzed in this research. A comprehensive analysis is also formulated between different techniques and accuracies.

Figure 1: report by International Journal of Multidisciplinary sciences and Engineering

The paper "Machine Learning Techniques for Sentiment Analysis: A Review" takes a detailed look at the various methods and algorithms used for sentiment analysis. It organizes these approaches into three main categories: lexicon-based, machine learning-based, and hybrid methods that combine features of both. The study highlights the growing importance of sentiment analysis in understanding user-generated content such as social media posts, reviews, and blogs, which are often used for analyzing opinions and detecting sentiment polarity.

Several machine learning techniques are discussed in the paper, including Support Vector Machines (SVM), Naive Bayes, Random Forest, Maximum Entropy (MaxEnt), and Stochastic Gradient Descent (SGD). These supervised algorithms are especially useful for classifying sentiments as positive or negative. For tasks without labeled

data, unsupervised techniques like K-Means and Apriori Algorithm are mentioned, while semi-supervised methods are explored for situations where both labeled and unlabeled data are available. The paper also highlights the role of advanced deep learning models, such as Multilayer Perceptrons (MLP), Convolutional Neural Networks (CNNs), and Long Short-Term Memory (LSTM) networks, which are particularly effective in understanding complex patterns and sequential text.

The importance of preprocessing steps, such as tokenization, stemming, and lemmatization, is also emphasized, as they are essential for enhancing the accuracy of sentiment analysis models. The paper addresses common challenges, such as dealing with imbalanced datasets, interpreting context-dependent language, and adapting to the constantly evolving nature of social media content. Hybrid approaches, which combine the strengths of lexicon-based and machine learning methods, are suggested as promising solutions for improving accuracy and performance.

The study concludes that while traditional methods like SVM and Naive Bayes are widely used for their efficiency, deep learning models like LSTM provide superior results when dealing with complex and sequential text data. By comparing various algorithms and highlighting their strengths and limitations, the paper offers valuable insights for researchers and practitioners working to create more accurate and effective sentiment analysis systems (Ahmad, 2017).

2.3.2 Research 2: A Survey on Sentiment Analysis Methods, Applications, and Challenges

Artificial Intelligence Review (2022) 55:5731–5780
https://doi.org/10.1007/s10462-022-10144-1

Check for updates

A survey on sentiment analysis methods, applications, and challenges

Mayur Wankhade[1,2] · Annavarapu Chandra Sekhara Rao[1,2] · Chaitanya Kulkarni[1,2]

Published online: 7 February 2022
© The Author(s), under exclusive licence to Springer Nature B.V. 2022

Abstract
The rapid growth of Internet-based applications, such as social media platforms and blogs, has resulted in comments and reviews concerning day-to-day activities. Sentiment analysis is the process of gathering and analyzing people's opinions, thoughts, and impressions regarding various topics, products, subjects, and services. People's opinions can be beneficial to corporations, governments, and individuals for collecting information and making decisions based on opinion. However, the sentiment analysis and evaluation procedure face numerous challenges. These challenges create impediments to accurately interpreting sentiments and determining the appropriate sentiment polarity. Sentiment analysis identifies and extracts subjective information from the text using natural language processing and text mining. This article discusses a complete overview of the method for completing this task as well as the applications of sentiment analysis. Then, it evaluates, compares, and investigates the approaches used to gain a comprehensive understanding of their advantages and disadvantages. Finally, the challenges of sentiment analysis are examined in order to define future directions.

Keywords Sentiment analysis · Text analysis · Word embedding · Machine learning · Social media

Figure 2: research on sentiment analysis by Mayur Wankhede and this team

The report "A Survey on Sentiment Analysis Methods, Applications, and Challenges" takes a close look at the techniques, real-world uses, and challenges involved in sentiment analysis. It explains three main approaches: lexicon-based methods, machine learning techniques, and hybrid models that combine the benefits of both. The report also highlights the importance of preprocessing steps, feature extraction, and using word embeddings like Word2Vec, GloVe, and FastText to make sentiment analysis more accurate and effective.

The study explores different levels of sentiment analysis, including document-level, sentence-level, and aspect-level, which are suited for various types of tasks and datasets. It also discusses advanced techniques like Recurrent Neural Networks (RNNs), Convolutional Neural Networks (CNNs), and transformer models such as

BERT, which have greatly improved the ability to understand complex and detailed sentiments in text.

The paper examines how sentiment analysis is used in areas like healthcare, customer feedback, business intelligence, social media monitoring, and political opinion analysis. It also looks at common challenges, such as detecting sarcasm, dealing with imbalanced datasets, handling multiple languages, and understanding context-specific language. The report suggests ways to address these problems, such as creating better datasets, developing more advanced models, and tailoring methods for specific industries or applications.

Finally, the report compares the strengths and weaknesses of various techniques, with visual aids like tables and charts to make the information clearer. It serves as a helpful resource for researchers and professionals, offering insights into current trends, identifying gaps in knowledge, and suggesting ways to improve sentiment analysis in the future (Wankhade, 2022).

2.4 Existing Challenges

Sentiment analysis, while widely applied, faces several challenges that impact its accuracy and effectiveness. These challenges stem from the complexity of human language, the dynamic nature of digital content, and technical limitations. Below are some key issues:

- Models struggle to identify sarcasm and irony, where sentiments often contradict the literal meaning of the words.
- Accurate sentiment analysis depends heavily on context, as the same word can convey different sentiments based on its usage.
- Sentiment datasets often have an unequal distribution of positive, negative, and neutral samples, leading to biased predictions.
- Handling multiple languages and code-switched content requires advanced models capable of understanding cultural and linguistic nuances.
- Sentences that convey mixed emotions or conflicting sentiments pose a challenge for accurate classification.
- Constant updates are needed to adapt models to new slang, emojis, abbreviations, and other evolving language patterns.
- User-generated content often includes typos, informal language, and unstructured text, complicating data preprocessing and analysis.
- Models trained on one domain, such as movie reviews, may perform poorly when applied to a different domain, like product reviews or social media content.
- Sentiment analysis of personal opinions raises ethical concerns, especially when data is used without explicit consent.
- Identifying fabricated reviews, bot-generated sentiments, or manipulated opinions is a growing challenge.
- Advanced models, such as transformers, require significant computational resources, making real-time applications difficult for some users.

2.5 Dataset

2.6 Dataset Information

The dataset "Sentiment Analysis Tweet" from Hugging Face is a curated collection of tweets designed for sentiment analysis tasks. It contains text data extracted from Twitter, where each tweet is labeled with a sentiment, such as positive, negative, or neutral. The dataset is structured to help machine learning models understand and classify the emotions conveyed in short, informal text messages, making it particularly suitable for analyzing social media sentiment.

The dataset is organized into labeled samples, where each entry includes:

- Tweet text: The actual content of the tweet, often featuring informal language, abbreviations, and emojis.
- Sentiment label: The corresponding sentiment category, such as positive, negative, or neutral.
- Preprocessing: Some tweets may already be cleaned of unnecessary characters or prepared for text analysis, though further preprocessing may still be required.
- This dataset is widely used for training and evaluating machine learning models in sentiment analysis tasks, offering a practical resource for building systems that can understand and classify emotions from short textual content.

2.7 Dataset Background

Social media platforms, particularly Twitter, are abundant sources of real-time user-generated content. People frequently share their opinions, feelings, and experiences on such platforms, making them a valuable resource for sentiment analysis. This dataset specifically focuses on extracting and analyzing sentiment from tweets, capturing the unique linguistic patterns and informal language used in social media.

Twitter's brevity and dynamic content, such as hashtags, mentions, emojis, and abbreviations, pose unique challenges for sentiment analysis. The "Sentiment Analysis Tweet" dataset was developed to address these challenges by providing labeled examples of real-world tweets. It serves as a foundation for developing machine learning models capable of understanding the nuances of informal text and identifying the emotions conveyed by users.

This dataset's significance lies in its ability to reflect public sentiment on various topics, making it useful in areas such as brand monitoring, public opinion tracking, and understanding trends in social behavior. By leveraging this dataset, researchers and developers can create models that classify tweets effectively, helping businesses, organizations, and researchers derive actionable insights from social media data.

3 Solution

After gathering information and insights from research, this project aims to address the challenges of understanding how social media impacts mental health, especially in detecting patterns of negative emotions that contribute to issues like anxiety, depression, and low self-esteem. It employs three algorithms: Long Short-Term Memory (LSTM) networks, Naïve Bayes, and Logistic Regression for sentiment analysis on social media posts. These methods analyze text over time, understand context, and identify shifts in emotional states, aiding in spotting at-risk individuals and informing preventive mental health strategies. This approach helps identify individuals who may be at risk and provides insights into general social trends, making it easier to develop preventive actions and support mental health efforts.

3.1 Explanation of purposed solution

This project aims to assess the impact of social media on mental health through a sentiment analysis model. By analyzing the emotional content expressed in social media posts, the objective is to understand how these sentiments relate to mental well-being. The focus is on building a model that can accurately interpret emotions, such as happiness, sadness, or anger, as expressed in user-generated content.

The process begins with a pre-labeled dataset of social media posts categorized by emotions. The data undergoes preprocessing to remove irrelevant characters, ensuring it is clean and ready for analysis. A machine learning model, trained specifically for sentiment classification, is then used to categorize the posts based on their emotional content. Each post is assigned a label that reflects the sentiment it conveys, enabling a deeper understanding of the patterns in emotional expressions.

This project emphasizes the ethical handling of data, adhering to strict privacy standards throughout its development. The dataset is used solely for training and testing the model, with no visualizations or external representations created. The goal is to provide a tool for researchers and mental health professionals to better understand the emotional impact of social media. This understanding could help develop strategies to mitigate the negative psychological effects associated with social media use.

3.2 Explanation of the algorithms used

3.2.1 Long short-term memory (LSTM)

Long Short-Term Memory (LSTM) is a type of neural network designed to process sequential data, such as text or time series, by remembering important details over long periods. Unlike traditional RNNs, which often struggle to retain earlier information in a sequence, LSTMs use a unique structure with memory cells and gates input, forget, and output to manage information flow. These gates help the model focus on relevant details and discard unnecessary ones, enabling it to understand the context and flow of data effectively. This makes LSTMs particularly useful for tasks like sentiment analysis, language processing, and time-series predictions, where understanding the sequence of events is critical (Saxena, 2024).

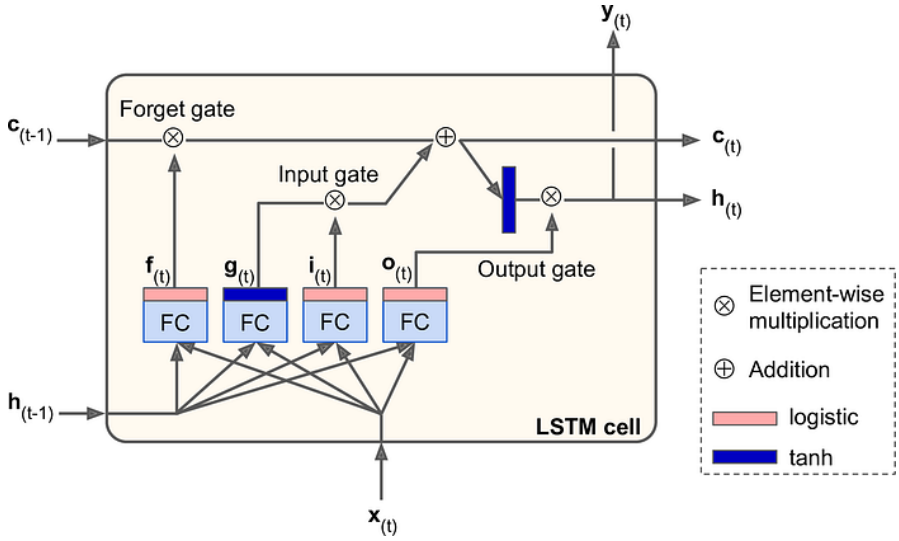


Figure 3: Architecture of an LSTM

The process begins with preparing a labeled dataset where each text sample is associated with a sentiment label, such as positive, negative, or neutral. The text data undergoes preprocessing to remove unnecessary characters like punctuation, emojis, or special symbols. Tokenization is then performed to convert the text into sequences of words, which are subsequently padded to ensure uniform input length. Using techniques like Word2Vec, GloVe, or an embedding layer, these words are transformed into numeric vectors that capture their semantic meaning, forming the input for the LSTM model.

The LSTM model processes the sequences step-by-step. For each word in the sequence, the input gate decides how much new information to allow into the memory cell, the forget gate determines which information to discard, and the output gate

regulates what information to pass to the next layer. This mechanism ensures that the model retains relevant context and long-term dependencies. During training, the model adjusts its internal weights using backpropagation through time (BPTT) to minimize the error between predicted and actual labels.

To demonstrate the process with a simple calculation, consider a small dataset:

Sentence	Label
"I love this"	Positive
"I hate this"	Negative
"It is okay"	Neutral

Table 1: sample dataset for example of LSTM

After preprocessing and tokenization, the vocabulary might look like this:

['I', 'love', 'this', 'hate', 'it', 'is', 'okay'].

Each word is assigned an embedding vector, such as:

- I → [0.1, 0.2, 0.3]
- love → [0.4, 0.5, 0.6]

The input sequence "I love this" becomes:

[[0.1, 0.2, 0.3], [0.4, 0.5, 0.6], [0.7, 0.8, 0.9]]

The LSTM processes the sequence as follows:

Input Gate: Determines how much of the new information to store:

$$\boldsymbol{it = \sigma(Wi \cdot [ht - 1, xt] + bi)}$$

Where $\boldsymbol{Wi}$ and $\boldsymbol{bi}$ are weights and biases, $\boldsymbol{xt}$ is the current input, and $\boldsymbol{ht - 1}$ is the previous hidden state.

Forget Gate: Decides what to forget from the previous cell state:

$$\boldsymbol{ft = \sigma(Wf \cdot [ht - 1, xt] + bf)}$$

Cell State Update: Updates the cell state based on input and forget gates:

$$\boldsymbol{Ct = ft * Ct - 1 + it * Ct}$$

Where $\boldsymbol{Ct}\sim\backslash\boldsymbol{tilde}\{\boldsymbol{C_t}\}Ct$ is the candidate cell state.

Output Gate: Controls what to output from the current cell state:

$$\boldsymbol{ot = \sigma(Wo \cdot [ht - 1, xt] + bo)}$$

Hidden State: Produces the hidden state for the next time step:

$$\boldsymbol{ht = ot * tanh(Ct)}$$

After processing the entire sequence, the final hidden state represents the sentiment of the input, which is passed to a dense output layer. For example, the probabilities for the sentiment might be:

Positive: 0.75

Negative: 0.15

Neutral: 0.10

The model predicts the label with the highest probability, in this case, "Positive."

LSTM's ability to process sequences and retain long-term dependencies makes it a powerful tool for sentiment analysis. By following these steps and leveraging its architecture, the model efficiently classifies text data, providing meaningful insights into the sentiment of the input.

3.2.1.1 Pseudocode

```
START
  IMPORT Libraries: NumPy, pandas, TensorFlow, Keras
  LOAD Training Dataset
  PRE-PROCESS Dataset:
    CONVERT text to lowercase
    REMOVE punctuation and numbers
    TOKENIZE text into words
    REMOVE stopwords
    PAD sequences to equal length
  SPLIT Data into training and testing sets
  CREATE LSTM Model:
    DEFINE input layer with appropriate input shape
    ADD LSTM layer with specified number of units
    ADD Dense layer with activation function (e.g., 'softmax' for multi-class classification)
  COMPILE Model:
    SET optimizer (e.g., 'adam')
    SET loss function (e.g., 'categorical_crossentropy')
    SET metrics (e.g., 'accuracy')
  TRAIN Model with training data and validation data
  DEFINE predict_text function:
    DO
      INPUT text
      PRE-PROCESS and VECTORIZE text
      APPLY Model to predict sentiment
      RETURN predicted sentiment label
    END DO
  EVALUATE Model on test data
  PRINT accuracy and loss
END
```

3.2.2 Logistic Regression

Logistic Regression is a supervised learning algorithm widely used for binary classification problems, including sentiment analysis. It predicts the probability that an input belongs to a particular class (e.g., positive or negative sentiment) using a logistic function, also known as the sigmoid function. Unlike linear regression, which predicts continuous values, logistic regression is specifically designed for categorical outcomes (Towards AI, 2021).

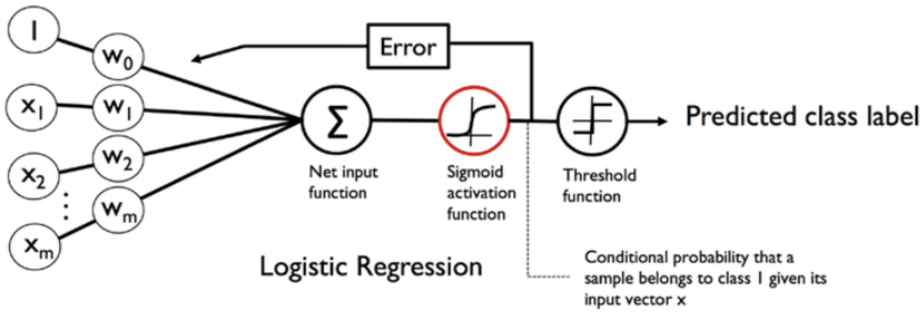


Figure 4: Architecture of a Logistic Regression Model

The process begins with preparing a labeled dataset where each text sample is associated with a sentiment label, such as positive or negative. The text data undergoes preprocessing to remove unnecessary characters like punctuation, emojis, or special symbols, and is then tokenized into words. Using methods like Bag of Words (BoW) or Term Frequency-Inverse Document Frequency (TF-IDF), these words are transformed into numerical vectors that represent their frequency or importance within the dataset. These numeric vectors serve as the input features for the Logistic Regression model.

The algorithm works by finding a linear relationship between input features and the log-odds of the target class. It calculates the probability of a class using the sigmoid function:

$$\boldsymbol{P(y = 1 \mid x) = 1 + e - (w \cdot x + b)1}$$

Here, w represents the weights, x the input features, and b the bias term. During training, the model learns the optimal values for www and b by minimizing the log-loss function, which measures the error between the predicted probabilities and the

actual labels. The process adjusts the weights iteratively using optimization techniques like Gradient Descent, ensuring the model improves its predictions over time.

To demonstrate the process with a simple calculation, consider a small dataset:

Sentence	Label
"I love this"	Positive
"I hate this"	Negative
"It is okay"	Neutral

Table 2: sample dataset for example of Logistic Regression

After preprocessing and vectorization using BoW, the vocabulary might look like this: ['I', 'love', 'this', 'hate', 'it', 'is', 'okay'].

Each sentence is converted into a vector:

- "I love this" → [1, 1, 1, 0, 0, 0, 0]
- "I hate this" → [1, 0, 1, 1, 0, 0, 0]
- "It is okay" → [0, 0, 0, 0, 1, 1, 1]

Assume the weights are initialized as:
W = [0.2,0.3,0.5,0.1,0.4,0.2,0.6]
b = 0.1

For the sentence "I love this," the model calculates:
z=(0.2·1)+(0.3·1)+(0.5·1)+(0.1·0)+(0.4·0)+(0.2·0)+(0.6·0)+0.1
z=0.2+0.3+0.5+0.1=1.1z = 0.2 + 0.3 + 0.5 + 0.1 = 1.1z=0.2+0.3+0.5+0.1=1.1

The probability is computed using the sigmoid function:
P(y=1|x)=1+e−1.11≈0.75

since the probability exceeds 0.5, the model predicts "Positive" for this input.

After training, the model evaluates its performance on new data by predicting probabilities and assigning labels based on the threshold of 0.5. For example, for the input "I hate this," the model calculates a lower probability for the positive class, resulting in the prediction "Negative."

Logistic Regression's simplicity and effectiveness make it an excellent choice for sentiment analysis, especially for datasets where the relationships between features

and outcomes are linear. Its ability to calculate probabilities and provide clear predictions makes it a reliable tool for classifying sentiments in text data.

3.2.2.1 Pseudocode of Logistic Regression

```
START
  IMPORT Libraries: NumPy, pandas, scikit-learn
  LOAD Training Dataset
  PRE-PROCESS Dataset:
    CONVERT text to lowercase
    REMOVE punctuation and numbers
    TOKENIZE text into words
    REMOVE stopwords
    VECTORIZE text using TF-IDF or CountVectorizer
  SPLIT Data into training and testing sets
  INITIALIZE Logistic Regression Model:
    SET up scikit-learn's LogisticRegression() class
    CONFIGURE parameters as necessary (e.g., C=1.0, max_iter=100)
  TRAIN Model:
    FIT the model using training data
  DEFINE predict_text function:
    DO
      INPUT text
      PRE-PROCESS and VECTORIZE text
      APPLY Model to predict sentiment
      RETURN predicted sentiment label
    END DO
  EVALUATE Model:
    CALCULATE and PRINT accuracy, precision, recall, and F1-score on test data
END
```

3.2.3 Naive Bayes

Naive Bayes is a probabilistic machine learning algorithm commonly used for text classification tasks, including sentiment analysis. It is based on Bayes' Theorem and assumes that features are independent given the class label. Despite this simplifying assumption, Naive Bayes performs well in practice, especially for text data, due to its computational efficiency and ability to handle high-dimensional inputs (Roy, 2021).

The process begins with preparing a labeled dataset where each text sample is associated with a sentiment label, such as positive, negative, or neutral. The text data undergoes preprocessing to remove unnecessary characters like punctuation, emojis, or special symbols, and is then tokenized into words. Using methods like Bag of Words (BoW) or Term Frequency-Inverse Document Frequency (TF-IDF), these words are converted into numerical representations that capture their frequency or importance.

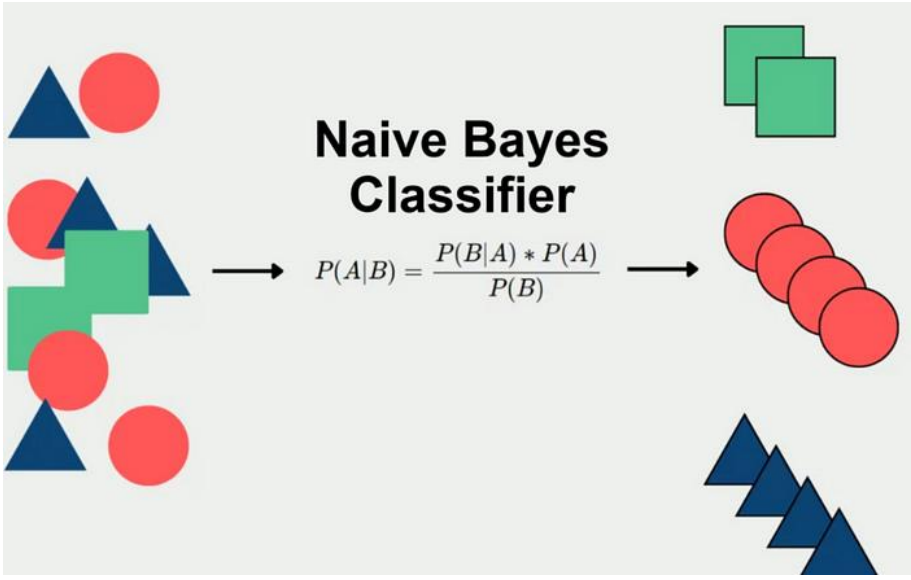


Figure 5: how naive bayes algorithm works

These numeric vectors serve as the input features for the Naive Bayes model.

Naive Bayes calculates the conditional probability of each class given the input features using Bayes' Theorem:

$$P(class|features)=P(features)P(features|class)\cdot P(class)$$

Since the denominator

P(features) is constant for all classes, it is ignored during classification. The algorithm predicts the class with the highest posterior probability.

To demonstrate the process with a simple calculation, consider a small dataset:

Sentence	Label
"I love this"	Positive
"I hate this"	Negative
"It is okay"	Neutral

Table 3: sample dataset for example of Naive Bayes

After preprocessing and vectorization using BoW, the vocabulary might look like this: ['I', 'love', 'this', 'hate', 'it', 'is', 'okay'].

Each sentence is converted into a vector:

- "I love this" → [1, 1, 1, 0, 0, 0, 0]
- "I hate this" → [1, 0, 1, 1, 0, 0, 0]
- "It is okay" → [0, 0, 0, 0, 1, 1, 1]

Assume the prior probabilities for each class are:

$$\boldsymbol{P(Positive) = 2/6, P(Negative) = 3/6, P(Neutral) = 1/6}$$

The algorithm calculates the likelihood of each word for each class using the formula:

$$\boldsymbol{P(word \mid class) = total\ words\ in\ class + vocabulary\ sizecount\ of\ word\ in\ class + 1}$$

For the class "Positive":

- Total words in positive sentences = 5
- Vocabulary size = 7

$$\boldsymbol{P(I \mid Positive) = 5 + 72 + 1 = 0.25}$$

$$\boldsymbol{P(love \mid Positive) = 5 + 71 + 1 = 0.17}$$

For the class "Negative":

- Total words in negative sentences = 6

$$\boldsymbol{P(I \mid Negative) = 6 + 73 + 1 = 0.31}$$

$$\boldsymbol{P(hate \mid Negative) = 6 + 71 + 1 = 0.15}$$

Classification Example

For the sentence "I love this," the probabilities are calculated as:
Positive:

$$\boldsymbol{P(Positive) \cdot P(I \mid Positive) \cdot P(love \mid Positive) \cdot P(this \mid Positive)}$$
$$\boldsymbol{= 0.33 \cdot 0.25 \cdot 0.17 \cdot 0.17 \approx 0.0024}$$

Negative:

$$\boldsymbol{P(Negative) \cdot P(I \mid Negative) \cdot P(love \mid Negative) \cdot P(this \mid Negative)}$$
$$\boldsymbol{= 0.5 \cdot 0.31 \cdot 0.15 \cdot 0.15 \approx 0.0035}$$

The model predicts "Negative" because it has the highest probability.

Naive Bayes efficiently classifies text data by leveraging probabilities derived from the training dataset. Its simplicity, speed, and effectiveness, especially with clean and structured data, make it an excellent choice for sentiment analysis. By following these steps and using labeled data, Naive Bayes provides meaningful insights into sentiment trends in text data.

3.2.3.1 Pseudocode of Naive Bayes

```
START
  IMPORT Libraries: NumPy, pandas, scikit-learn
  LOAD Training Dataset
  PRE-PROCESS Dataset:
    CONVERT text to lowercase
    REMOVE punctuation and numbers
    TOKENIZE text into words
    REMOVE stopwords
  SPLIT Data into training and testing sets
  VECTORIZE Text using CountVectorizer
  INITIALIZE Naïve Bayes Classifier
  TRAIN Classifier with training data
  DEFINE predict_text function:
    DO
      INPUT text
      PRE-PROCESS and VECTORIZE text
      CALCULATE probabilities for each class (positive, neutral, negative)
      DETERMINE sentiment based on highest probability
      RETURN sentiment
    END DO
  APPLY predict_text on test data
  EVALUATE model performance
  PRINT results
END
```

3.3 Diagrammatical representations of the solution

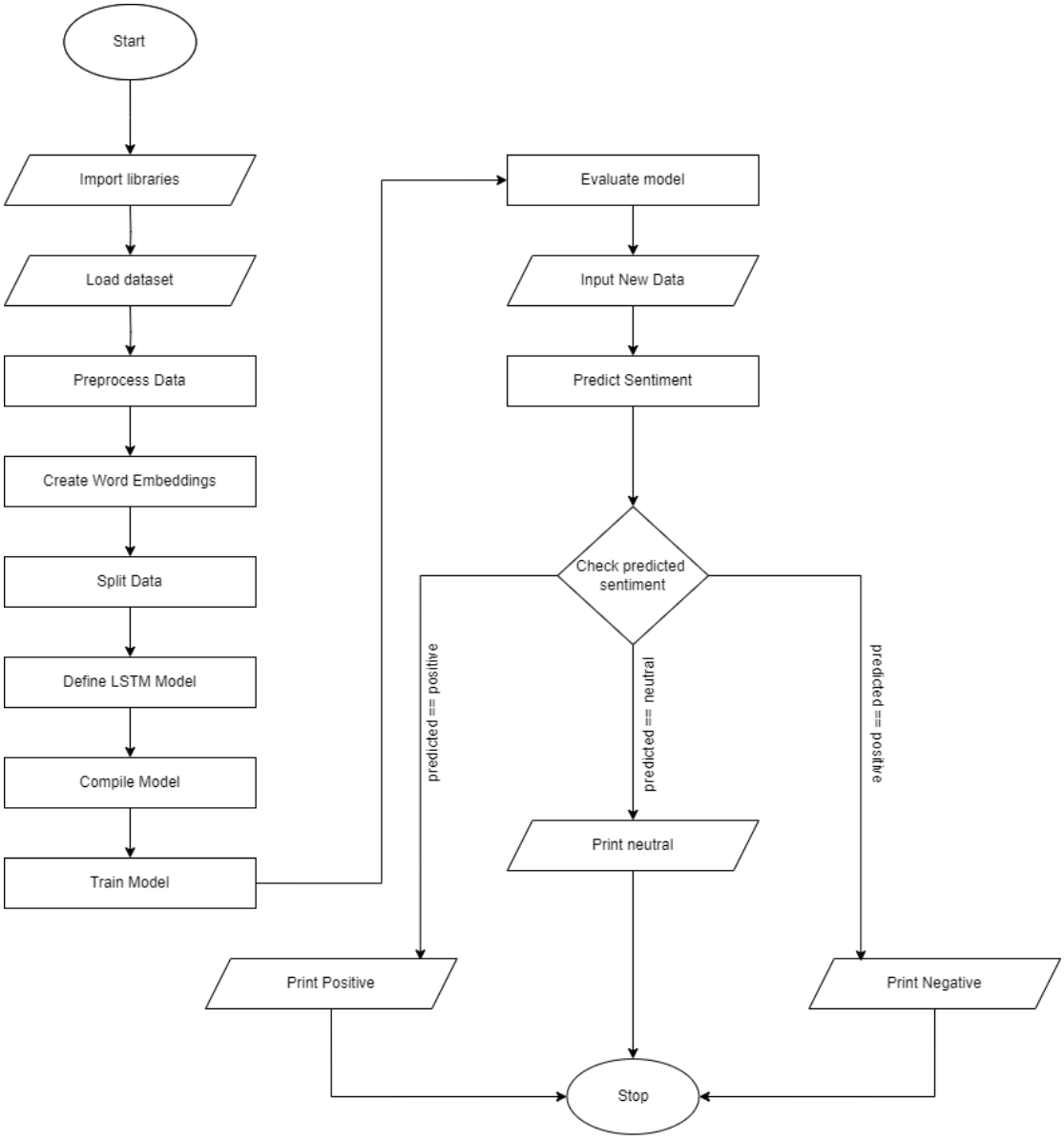


Figure 6: flowchart of LSTM model

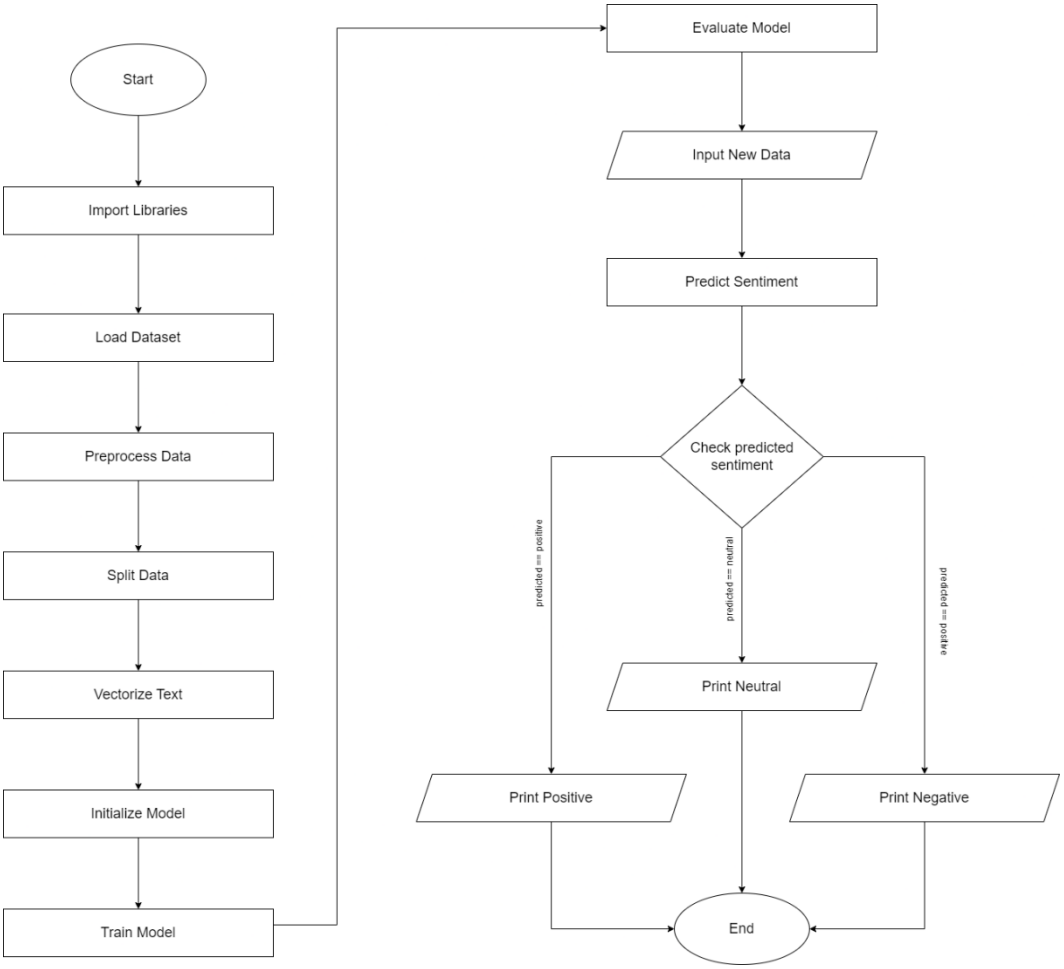


Figure 7: Flowchart of Logistic Regression

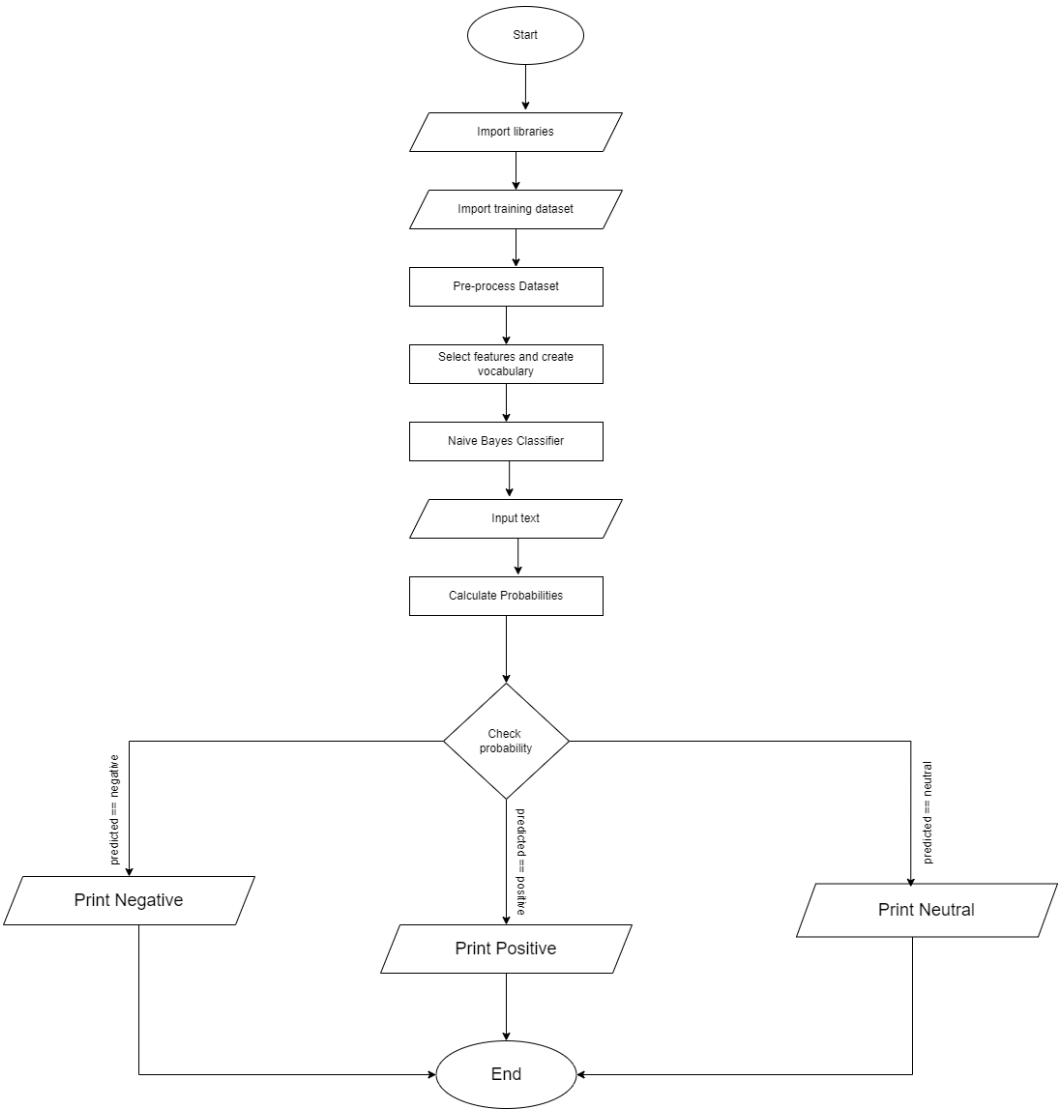


Figure 8: Flowchart of Naive Bayes

3.4 Development process requirements

This section describes the methodical, multi-step strategy employed in the project's development, aimed at enhancing data processing efficiency and building precise AI models. It involved choosing suitable tools, technologies, and libraries to facilitate seamless progress through each phase and deliver optimal outcomes. Below is a comprehensive outline of all the crucial stages involved, accompanied by detailed explanations:

3.4.1 Libraries used:

- Pandas

  Pandas is a staple in data manipulation and analysis, providing rich data structures and functions designed to make working with structured data fast, flexible, easy, and expressive. Primarily, it offers DataFrame objects that are like spreadsheets but with much more power and flexibility. It's particularly well-suited for handling and cleaning large datasets, enabling you to carry out everything from merging and reshaping data, to summarizing it in tables or through descriptive statistics (pypi, 2024).

Figure 9: pandas logo

- NumPy

  NumPy is fundamental for scientific computing with Python. It provides support for large, multi-dimensional arrays and matrices, along with a vast library of high-level mathematical functions to operate on these arrays. The ability to perform complex mathematical operations with incredible speed and the tool's integration into nearly every other data science or machine learning library makes it indispensable

for numerical computations, transforming lists of numbers into meaningful insights (NumPy , 2024).

Figure 10: NumPy logo

- Matplotlib

  Matplotlib is the grandfather of Python visualization libraries, despite newer tools in the market. It provides an extensive array of plotting functions to create standard scientific plots, interactive visualizations, and detailed graphs. Whether you're plotting histograms, scatter plots, or line charts, Matplotlib gives you the tools to provide clear and actionable insights into data. It's highly customizable and works well with other libraries like Pandas and NumPy, making it a reliable tool for exploratory data analysis (Matplotlib, 2025).

Figure 11: Matplotlib logo

- Scikit-Learn

  Scikit-Learn is designed to interoperate with the Python numerical and scientific libraries NumPy and SciPy. It's one of the most used libraries for general machine learning, offering a range of supervised and unsupervised learning algorithms via a consistent interface in Python. It simplifies complex processes like fitting models, making predictions, and tuning parameters, which are essential for building, analyzing, and validating complex models with minimal code (pypi, 2024).

Figure 12: Scikit Learn logo

- TensorFlow

  TensorFlow is a powerful library for deep learning, a subset of machine learning where models capable of making complex predictions are trained over large datasets. Created by researchers and engineers in the Google Brain team, it allows developers to create large-scale neural networks with many layers. TensorFlow is highly versatile and supports both CPUs and GPUs. It is also equipped with an intuitive construct called a Tensor Board, which allows users to visually monitor the training and performance of neural networks as they learn (Tensor Flow, 2024).

Figure 13: tensor flow logo

3.5 Explanation of the development process

Development process of the AI application involves several steps, the explanation of development process are mentioned below:

3.5.1 Importing libraries

```python
# Core Data Manipulation and Numerical Computing
import pandas as pd
import numpy as np
import math
import time

# Machine Learning and Model Selection
from sklearn.model_selection import train_test_split
from sklearn.utils import resample

# Advanced Metrics and Evaluation
from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    accuracy_score,
    f1_score,
    mean_squared_error,
    mean_absolute_error,
    precision_score,
    recall_score,
    hamming_loss
)

# Visualization Libraries
import matplotlib.pyplot as plt
import seaborn as sns

# Deep Learning and Neural Network Tools
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import (
    EarlyStopping,
    ModelCheckpoint,
    ReduceLROnPlateau,
    LearningRateScheduler
)

# Advanced Keras Layers and Regularization
from tensorflow.keras.layers import (
    Embedding,
    SpatialDropout1D,
    Conv1D,
    BatchNormalization,
    MaxPooling1D,
    Bidirectional,
    LSTM,
    Dense,
    Dropout,
    Flatten
)
from tensorflow.keras.regularizers import l2

# Text and Feature Processing
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.utils.class_weight import compute_class_weight
```

```python
# Classical Machine Learning Classifiers
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import MultinomialNB

# Imbalanced Data Handling
from imblearn.over_sampling import SMOTE

# Natural Language Processing
import re
import nltk
nltk.download('punkt')
from nltk.tokenize import word_tokenize
```

Figure 14: import statements

3.5.2 Setting up the data that will be used later for training the ai model

Here data is loaded in data frame from dataset for training model.

Data Collection

```
# path of dataset
train_dataset_path = '/content/train.csv'
test_dataset_path = '/content/test.csv'
```

```
#loading data in pandas dataframe
test_df = pd.read_csv(test_dataset_path)
train_df = pd.read_csv(train_dataset_path)
```

```
# merging both train and test for preprocessing
df = pd.concat([train_df, test_df], axis=0)
```

```
# viewing dataset sample
df.head()
```

	tweet	label
0	BofA previews Netflixs NFLX Q3 Earnings Tues 0...	0
1	I scooped a couple of shares this morning at a...	0
2	Im streaming ES Futures using Bookmap on youtu...	0
3	CF taking some off here close to 19150	1
4	No change to this position is still bullish st...	0

Figure 15: loading data for from dataset and checking data frame

3.5.3 Preprocessing data

In the data preprocessing phase, we rigorously clean and standardize the dataset to optimize the performance of our machine learning model. This involves removing extraneous whitespaces and hyperlinks, which can introduce noise and inconsistencies, standardizing text to a uniform case to ensure all words are treated equally and stripping out non-alphabetic characters that are irrelevant to analysis. Additionally, tokenization breaks text into manageable pieces, and common stop words that offer little analytical value are eliminated. These steps collectively help in creating a clean dataset that enhances model training efficiency and accuracy by focusing only on meaningful and informative features.

Data Preprocessing

```
#Looking shape of df
df.shape
```

```
(14913, 2)
```

```
#Lo0king for number of rows for each label
print("total number of negative sentiment labels: ", (df['label'] == 1).sum())
print("total number of neutral sentiment labels: ",(df['label'] == 0).sum())
print("total number of positive sentiment labels: ",(df['label'] == -1).sum())
```

```
total number of negative sentiment labels:  3809
total number of neutral sentiment labels:  7413
total number of positive sentiment labels:  3691
```

```
# Setting up data t make graphs
sentiments = ['Negative', 'Neutral', 'Positive'] # Labels
counts = [(df['label'] == -1).sum(), (df['label'] == 0).sum(), (df['label'] == 1).sum()] # numbers of each labels
colors = ['#ff6b6b', '#4dabf7', '#51cf66']  # Red, Blue, Green for easy understanding of graph
```

```
# Calculating percentages for labels
total = sum(counts)
percentages = [count/total * 100 for count in counts]
```

```
# Setting up the figure with two subplots side by side
plt.figure(figsize=(15, 7))
```

```
<Figure size 1500x700 with 0 Axes>
```

```
<Figure size 1500x700 with 0 Axes>
```

```
# Bar Chart

plt.plot()
bars = plt.bar(sentiments, counts, color=colors)

# Customize bar chart
plt.title('Sentiment Distribution', pad=15, fontsize=12, fontweight='bold')
plt.ylabel('Number of Texts', fontsize=10)
plt.grid(axis='y', linestyle='--', alpha=0.7)

# Add value labels on top of bars
for bar in bars:
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, height,
             f'{int(height):,}',  # Format with commas for thousands
             ha='center', va='bottom', fontsize=10)
```

Figure 16: looking of imbalance dataset

3.5.4 Looking for possible imbalance in dataset labels

Handling imbalanced labels in datasets is crucial for building effective machine learning models, particularly in classification tasks where the disparity in class frequencies can lead to biased predictions toward the majority class. Techniques like resampling the minority class, adjusting class weights, and employing ensemble methods are commonly used to address this issue. It's also important to rely on metrics like precision, recall, and the F1-score instead of accuracy to evaluate model performance, as these provide more insight into the model's ability to correctly predict minority class instances. Employing strategies like stratified cross-validation ensures that each fold reflects the overall class distribution, aiding in more accurate model assessment and robustness.

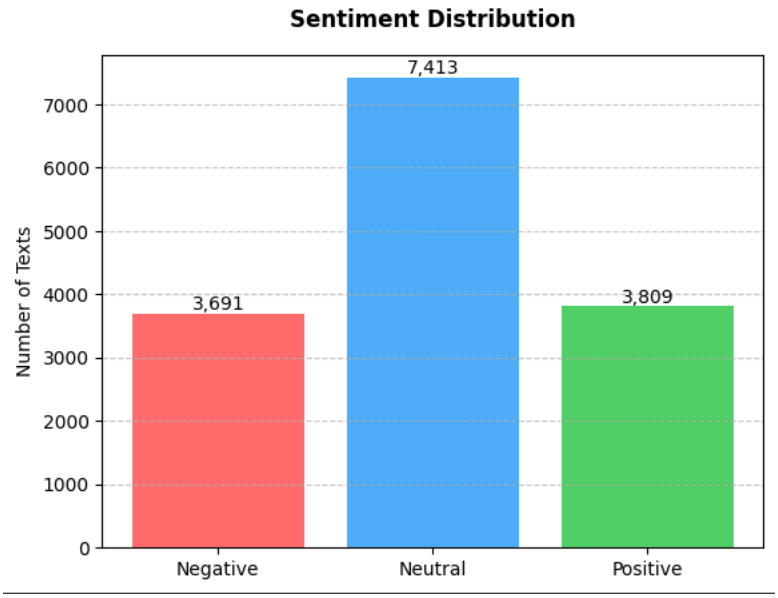


Figure 17: graph showing imbalance in dataset

```
# to balance the dataset I am choosing the oversanpling

# calculating max
max_size = df['label'].value_counts().max()
```

```
# initializing empty list to add new over sampled datasets
over_sampled = []
```

```
# over sampling datset to make all labels balanced
for class_index, group in df.groupby('label'):
    over_sampled.append(resample(group,
                                 replace=True,
                                 n_samples=max_size,
                                 random_state=123))
```

```
# setting up final balanced dataset
final_dataset = pd.concat(over_sampled)
```

Figure 18: over sampling dataset to make labels balanced

3.5.5 Cleaning Dataset

```
# Removing HTML tags
final_dataset['tweet'] = final_dataset['tweet'].apply(lambda x: re.sub(r'<.*?>', '', x))

# Removing non-alphabetic characters
final_dataset['tweet'] = final_dataset['tweet'].apply(lambda x: re.sub(r'[^a-zA-Z\s]', '', x))

# Converting text to lowercase
final_dataset['tweet'] = final_dataset['tweet'].apply(lambda x: x.lower())

# Removing URLs
final_dataset['tweet'] = final_dataset['tweet'].apply(lambda x: re.sub(r'http\S+|www\S+|https\S+', '', x))

# Removing user mentions
final_dataset['tweet'] = final_dataset['tweet'].apply(lambda x: re.sub(r'@\w+', '', x))

# Removing numbers
final_dataset['tweet'] = final_dataset['tweet'].apply(lambda x: re.sub(r'\d+', '', x))

# Removing extra spaces
final_dataset['tweet'] = final_dataset['tweet'].apply(lambda x: re.sub(r'\s+', ' ', x).strip())
```

```
final_dataset.head()
```

	tweet	label
4476	if she was a white male saying particularly wh...	-1
3877	shorts in my book closed short term long all c...	-1
3936	min line break n break down then tailing candl...	-1
5603	this is going to have a lot of ramifications b...	-1
4742	yes sadly i think it has to go horribly too fa...	-1

Figure 19: preprocessing data

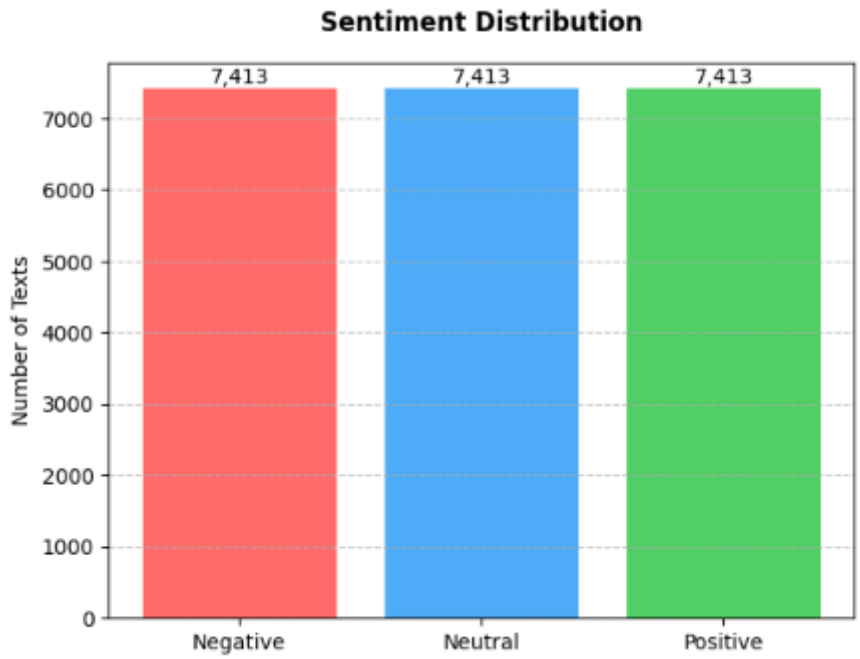


Figure 20: graph showing balanced label after over sampling

3.5.6 Split the Data

Splitting a dataset into training and test sets is a critical process in machine learning. It ensures that the model is trained on one subset of the data (training set) and evaluated on a separate subset (test set). This practice helps verify that the model can generalize well to new data, not just perform well on the data it was trained on. It's essential for preventing overfitting and ensuring the model's reliability in real-world applications.

```
# splitting dataset for train and test
X_train, X_test, y_train, y_test = train_test_split(final_dataset['tweet'], final_dataset['label'] + 1, test_size=0.25, random_state=42)
```

```
# Converting y_train to numpy array
y_train = np.array(y_train)
```

Figure 21: splitting datasets for training and testing

```
# Preprocessing
max_features = 10000
sequence_length = 250

# Tokenization
tokenizer = Tokenizer(num_words=max_features, oov_token='<OOV>')
tokenizer.fit_on_texts(X_train)

# Convert to sequences
X_train_sequences = tokenizer.texts_to_sequences(X_train)
X_test_sequences = tokenizer.texts_to_sequences(X_test)

# Pad sequences
X_train_padded = pad_sequences(X_train_sequences, maxlen=sequence_length, padding='post', truncating='post')
X_test_padded = pad_sequences(X_test_sequences, maxlen=sequence_length, padding='post', truncating='post')
```

Figure 22: tokenizing text to train in LSTM model

```
vectorizer = TfidfVectorizer(stop_words='english', max_features=20000)
X_train_features = vectorizer.fit_transform(X_train)
X_test_features = vectorizer.transform(X_test)
```

Figure 23: vectorizing text to train in Naive bayes and Logistic Regression

3.5.7 Select Model and training Model

Selecting the right model for a machine learning task involves considering the nature of the data, the specific problem to be solved, and the desired outcome. The choice is influenced by factors such as the accuracy, interpretability, and computational efficiency of potential models. It often begins with simpler models to establish a baseline and progresses to more complex models if necessary. Experimentation and cross-validation are used to compare the performance of various models, ensuring the final choice is well-suited to generalizing on unseen data effectively.

```
# Compute class weights
class_weights = compute_class_weight(
    class_weight='balanced',
    classes=np.unique(y_train),
    y=y_train
)
class_weights = dict(enumerate(class_weights))


def create_model():
    model = Sequential([
        Embedding(input_dim=max_features, output_dim=100, input_length=sequence_length),
        Bidirectional(LSTM(32, return_sequences=False, kernel_regularizer=l2(1e-5))),
        BatchNormalization(),
        Dropout(0.3),
        Dense(16, activation='relu', kernel_regularizer=l2(1e-5)),
        BatchNormalization(),
        Dropout(0.3),
        Dense(3, activation='softmax')
    ])

    return model

# Learning rate scheduler
def lr_schedule(epoch):
    lr = 1e-3
    if epoch > 10:
        lr *= 0.1
    if epoch > 20:
        lr *= 0.1
    return lr
```

Figure 24: Creating parameters for LSTM model

```python
# Callbacks
callbacks = [
    EarlyStopping(
        monitor='val_loss',
        patience=15,
        restore_best_weights=True
    ),
    ReduceLROnPlateau(
        monitor='val_loss',
        factor=0.2,
        patience=5,
        min_lr=1e-6
    ),
    ModelCheckpoint(
       'best_model.keras',
        monitor='val_loss',
        save_best_only=True
    ),
    LearningRateScheduler(lr_schedule)
]

# Create and compile the model
model = create_model()
optimizer = Adam(learning_rate=1e-3)
model.compile(
    optimizer=optimizer,
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# Train the model
history = model.fit(
    X_train_padded,
    y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.2,
    callbacks=callbacks,
    class_weight=class_weights
)
```

Figure 25: creating model and training LSTM model

Model Design

```
logistic_model = LogisticRegression(random_state=42)
```

```
nb_model = MultinomialNB()
```

Figure 26: setting model for Logistic regression and naive bayes

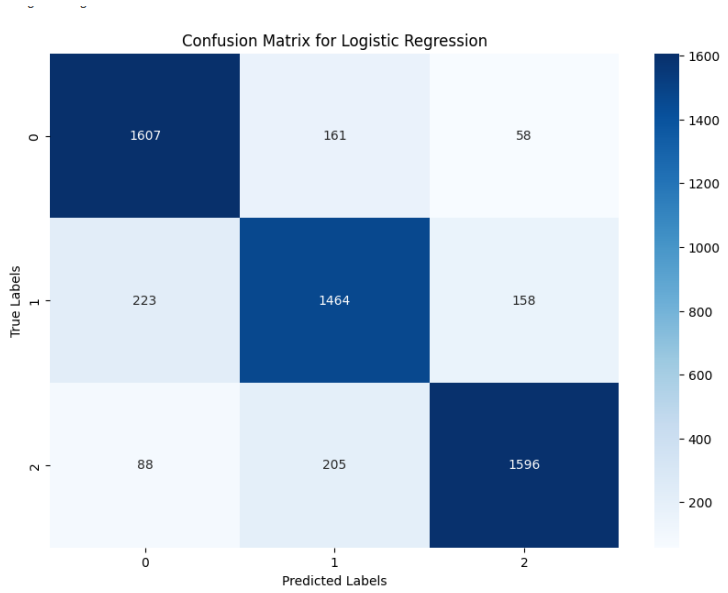
```
def train_and_evaluate(model, model_name):
    """
    Train and evaluate a machine learning model for multi-label classification.

    Parameters:
    model: the machine learning model to train and evaluate.
    model_name (str): the name of the model for display purposes.

    Prints the performance of the model including time taken, accuracy, error metrics, F1 score, and other classification metrics.
    """
    try:
        start_time = time.time()
        model.fit(X_train_features, y_train)
        y_pred = model.predict(X_test_features)
        end_time = time.time()
```

Figure 27: training Naive bayes and Logistic regression model

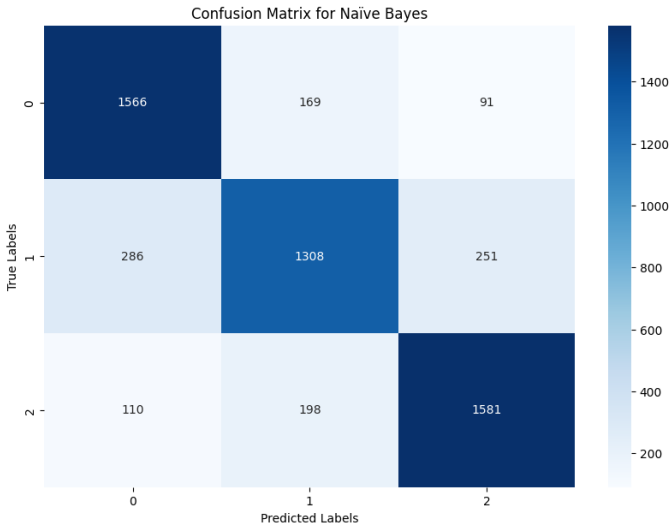
Evaluating Model

```
    # Evaluation Metrics for Multi-Label Classification
    accuracy = accuracy_score(y_test, y_pred)
    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_test, y_pred)
    f1 = f1_score(y_test, y_pred, average='macro')
    precision = precision_score(y_test, y_pred, average='macro')
    recall = recall_score(y_test, y_pred, average='macro')
    hamming = hamming_loss(y_test, y_pred)
    cm = confusion_matrix(y_test, y_pred)

    # Print Results
    print(f"\n{model_name} Model Performance:")
    print(f"Time Taken: {end_time - start_time:.4f} seconds")
    print(f"Accuracy: {accuracy:.2f}")
    print(f"Mean Squared Error (MSE): {mse:.2f}")
    print(f"Root Mean Squared Error (RMSE): {rmse:.2f}")
    print(f"Mean Absolute Error (MAE): {mae:.2f}")
    print(f"F1 Score: {f1:.2f}")
    print(f"Precision: {precision:.2f}")
    print(f"Recall: {recall:.2f}")
    print(f"Hamming Loss: {hamming:.2f}")
    print(f"Classification Report:\n{classification_report(y_test, y_pred)}")

    # Display Confusion Matrix
    plt.figure(figsize=(10, 7))
    sns.heatmap(cm, annot=True, fmt='g',  cmap='Blues')
    plt.title(f'Confusion Matrix for {model_name}')
    plt.xlabel('Predicted Labels')
    plt.ylabel('True Labels')
    plt.show()

except Exception as e:
    print(f"An error occurred while training or evaluating the {model_name} model: {str(e)}")
```

Figure 28: Evaluating Naive Bayes and Logistic Regression



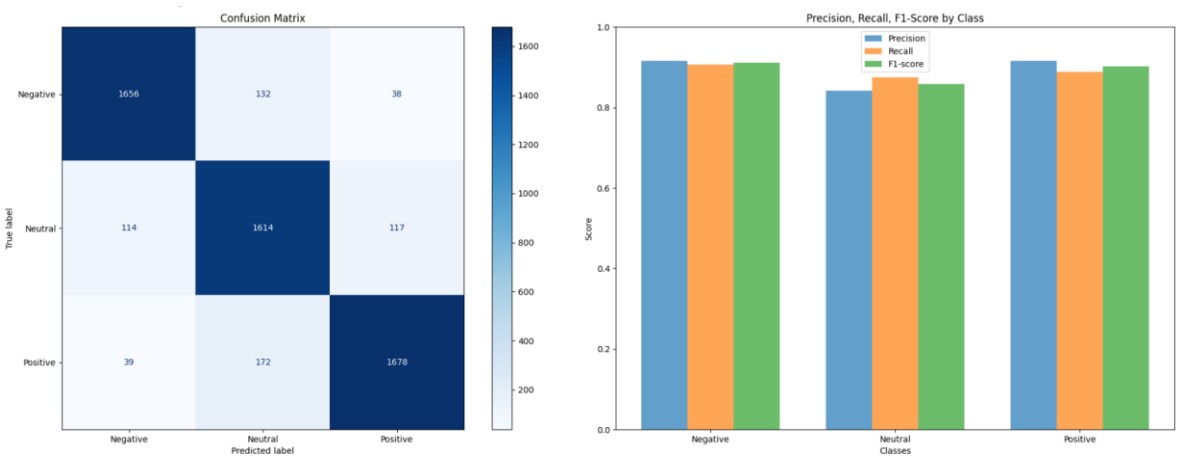
```
Logistic Regression Model Performance:
Time Taken: 3.4702 seconds
Accuracy: 0.84
Mean Squared Error (MSE): 0.24
Root Mean Squared Error (RMSE): 0.49
Mean Absolute Error (MAE): 0.19
F1 Score: 0.84
Precision: 0.84
Recall: 0.84
Hamming Loss: 0.16
Classification Report:
              precision    recall  f1-score   support

           0       0.84      0.88      0.86      1826
           1       0.80      0.79      0.80      1845
           2       0.88      0.84      0.86      1889

    accuracy                           0.84      5560
   macro avg       0.84      0.84      0.84      5560
weighted avg       0.84      0.84      0.84      5560
```

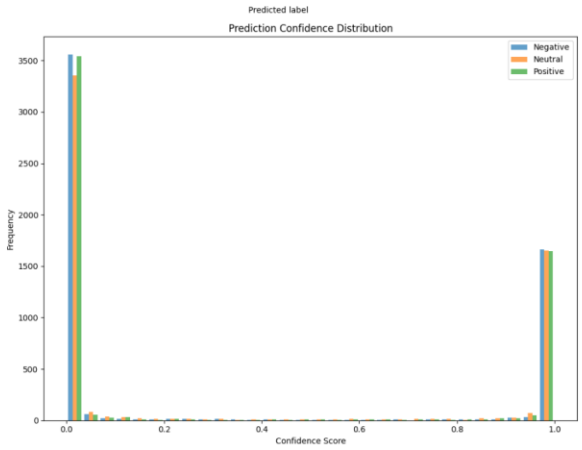


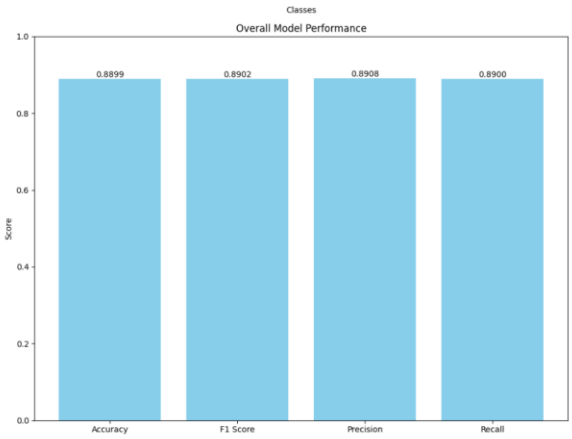

```
Naïve Bayes Model Performance:
Time Taken: 0.0262 seconds
Accuracy: 0.80
Mean Squared Error (MSE): 0.31
Root Mean Squared Error (RMSE): 0.55
Mean Absolute Error (MAE): 0.23
F1 Score: 0.80
Precision: 0.80
Recall: 0.80
Hamming Loss: 0.20
Classification Report:
              precision    recall  f1-score   support

           0       0.80      0.86      0.83      1826
           1       0.78      0.71      0.74      1845
           2       0.82      0.84      0.83      1889

    accuracy                           0.80      5560
   macro avg       0.80      0.80      0.80      5560
weighted avg       0.80      0.80      0.80      5560
```

```python
# Evaluate the model
y_pred = model.predict(X_test_padded)
y_pred_classes = np.argmax(y_pred, axis=1)

# Compute Metrics
accuracy = accuracy_score(y_test, y_pred_classes)
f1 = f1_score(y_test, y_pred_classes, average='macro')
precision = precision_score(y_test, y_pred_classes, average='macro')
recall = recall_score(y_test, y_pred_classes, average='macro')
cm = confusion_matrix(y_test, y_pred_classes)

# Detailed Reporting
plt.figure(figsize=(20, 15))

# 1. Confusion Matrix Visualization
plt.subplot(2, 2, 1)
ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Negative', 'Neutral', 'Positive']).plot(
    cmap='Blues', values_format='d', ax=plt.gca()
)
plt.title('Confusion Matrix', fontsize=12)

# 2. Precision, Recall, F1-Score Bar Chart
plt.subplot(2, 2, 2)
class_report = classification_report(y_test, y_pred_classes, target_names=['Negative', 'Neutral', 'Positive'], output_dict=True)

metrics = ['precision', 'recall', 'f1-score']
classes = ['Negative', 'Neutral', 'Positive']

bar_width = 0.25
index = np.arange(len(classes))

for i, metric in enumerate(metrics):
    values = [class_report[cls][metric] for cls in classes]
    plt.bar(index + i*bar_width, values, bar_width,
            label=metric.capitalize(), alpha=0.7)

plt.xlabel('Classes')
plt.ylabel('Score')
plt.title('Precision, Recall, F1-Score by Class')
plt.xticks(index + bar_width, classes)
plt.legend()
plt.ylim(0, 1)
```

Figure 29: evaluating LSTM model

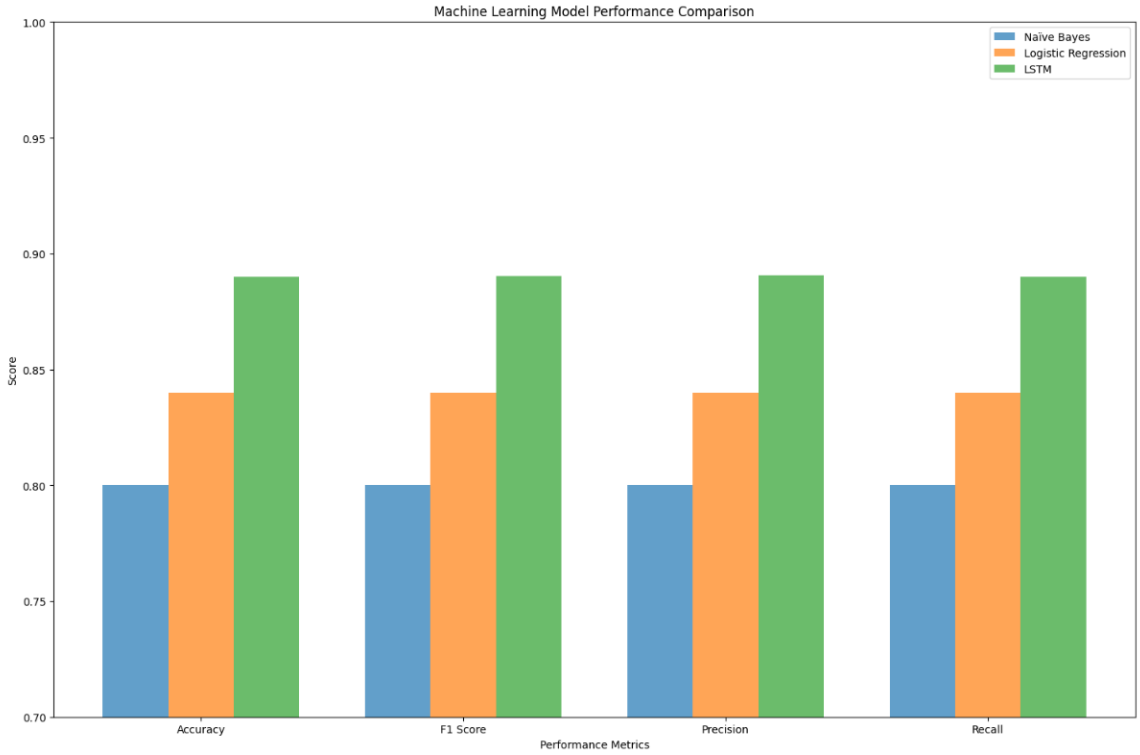



Predicted label
Prediction Confidence Distribution
Negative
Neutral
Positive
Frequency
Confidence Score


Classes
Overall Model Performance
0.8899
0.8902
0.8908
0.8900
Score
Accuracy
F1 Score
Precision
Recall

3.5.8 Comparison

Considering accuracy, precision, recall, and the F1 score collectively, the LSTM Model appears to be the best performer for this dataset. It offers a well-rounded performance with slightly better recall and similar F1 scores compared to Logistic Regression, indicating a robust ability to manage class imbalances and correctly identify class members across the dataset. This model would likely be the most effective choice for scenarios where a balanced approach to false positives and negatives is critical.



To use this model a website is created where user can add sentences either single sentence or bulk sentences and predict sentiments of those sentences.

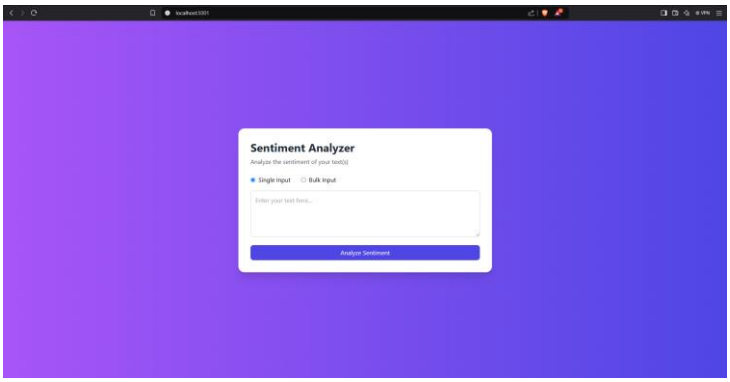


Figure 30: website developed to predict sentiments using best model that was developed

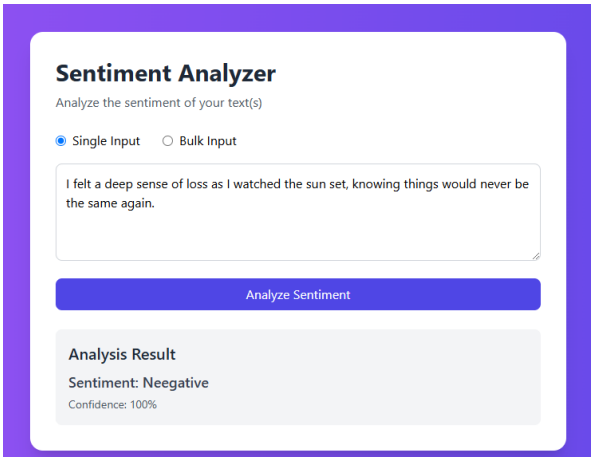


Figure 31: Predicting sentiments using developed website

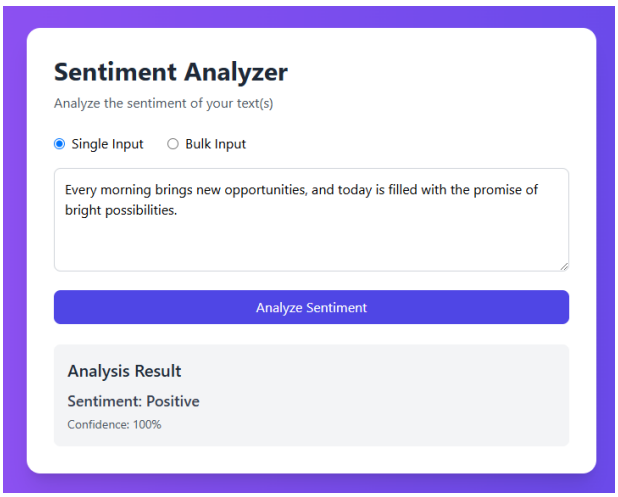


Figure 32: Predicting Sentiments

4 Conclusion

This project aims to analyze the emotional impact of social media on mental health using three algorithms: LSTM, Logistic Regression, and Naïve Bayes. By classifying social media posts into positive, negative, or neutral sentiments, the study provides insights into how these platforms influence emotions. The goal is to support mental health research and contribute to strategies that reduce the negative effects of social media, fostering a more supportive and healthier online environment.

4.1 Analysis of work done

Artificial Intelligence combines disciplines like philosophy, psychology, and linguistics with computer science to create advanced systems capable of tasks that typically require human intelligence. This proposal delves into sentiment analysis, a crucial component of **Natural Language Processing (NLP)**, which aims to develop applications that can accurately interpret human language.

Our research is centered on developing effective methods for analyzing sentiments expressed in text. This involves exploring various natural language processing techniques and their applications in creating systems that understand human emotions and opinions related to products, services, or events.

The need for systems capable of understanding and predicting human responses is vital, particularly for industries like business where consumer sentiment is directly linked to growth. Our proposal outlines strategies for utilizing extensive online data to develop predictive models that can effectively address and mitigate potential challenges.

This document provides a comprehensive overview of the research conducted on sentiment analysis, discussing the potential for NLP techniques to improve the accuracy and efficiency of sentiment detection systems.

S.N.	Task	Status
1	Research on AI	Complete
2	Research on Fake News Detecting System	Complete
3	Topic Exploration	Complete
4	Problem Statement of chosen topic	Complete
5	Research on work done method	Complete
6	Review and analysis of existing work	Complete
7	Explore of solution	Complete
8	Flow chart and pseudocode	Complete
9	Implementing code	Complete
10	Testing project	Complete

Table 4: Summary table of work analysis

In summary, this proposal offers a detailed exploration of sentiment analysis within AI, presenting researched strategies and potential solutions for advancing the understanding of human sentiment through automated systems. This groundwork aims to inspire continued innovation and application in this dynamic field.

4.2 How the solution addresses the real-world problems

In the digital era, social media has become a fundamental part of daily life, but its impact on mental health is significant. While these platforms connect us like never before, they also create environments where negative emotions can thrive, potentially intensifying issues like anxiety, depression, and low self-esteem among users. This research project recognizes the deep effect social media has on our emotional health and aims to address these issues with advanced technology.

Using sentiment analysis powered by Long Short-Term Memory (LSTM) networks, our project examines the large amounts of text data generated on social media platforms. LSTMs are especially good at reading sentences, understanding their meaning, and tracking how sentiments change over time. This ability lets us spot subtle changes in language and emotion, which can be early signs of mental health problems.

For example, a gradual increase in words that express sadness or negativity in someone's posts may indicate a worsening state of depression. By spotting these

patterns, our system can identify individuals who might be at risk. This proactive method not only improves our understanding of the wider social trends impacting mental health but also helps in creating effective preventative strategies and alternative treatments.

Ultimately, this project aims to use the power of artificial intelligence to provide insights that are proactive, not just reactive. We want to offer tools that can protect vulnerable people from the psychological effects of negative interactions on social media. By changing how we interpret and react to emotional signals in digital communications, we hope to make a positive impact on the mental well-being of society.

4.3 Further work

As we build on the work started with this research, we plan to make several improvements to help the project do more and reach further. These steps are designed to make our system better at understanding and responding to the emotions people express on social media.

- More Emotion Categories: Right now, we sort emotions into three groups. We want to add more categories, like 'mixed' or 'confused,' to better capture the range of feelings people show online. This will help us understand and respond more accurately to what people are going through.
- Connecting with Interactive Platforms: We’re looking to link our system with platforms that can interact directly with users. This would allow us to offer immediate support and advice to those who might be feeling down or at risk, based on what they post.
- Working with Mental Health Experts: By teaming up with professionals who understand mental health well, we can make sure our system notices the right signs of emotional distress. This collaboration will also help us give better advice to those who need it.
- Stronger Privacy Protections: Because we deal with people's personal feelings, we need to be very careful with the information we gather. We plan to strengthen how we protect data to keep everyone’s details safe and private.

- Long-Term Studies: We want to start studies that watch how people use social media over time and how it affects their mental health. This will help us see if our system is making a difference and find better ways to help in the future.

These steps will make our project more useful, safe, and effective at helping to understand and improve mental health through social media.

5 References

Ahmad, M., 2017. *Machine Learning Techniques for Sentiment.* [Online] Available at: https://www.researchgate.net/profile/Shabib-Aftab-2/publication/317284281_Machine_Learning_Techniques_for_Sentiment_Analysis_A_Review/links/59302d6ba6fdcc89e78431ec/Machine-Learning-Techniques-for-Sentiment-Analysis-A-Review.pdf
[Accessed 23 12 2024].

AWS, 2024. *What is Sentiment Analysis?.* [Online] Available at: https://aws.amazon.com/what-is/sentiment-analysis/#:~:text=Sentiment%20analysis%20is%20an%20application,before%20providing%20the%20final%20result.
[Accessed 22 12 2024].

Berger, M. W., 2018. *Social media use increases depression and loneliness.* [Online] Available at: https://penntoday.upenn.edu/news/social-media-use-increases-depression-and-loneliness?form=MG0AV3
[Accessed 21 12 2024].

Bergman, A. M., 2024. *Social Media and FOMO.* [Online] Available at: https://socialmediavictims.org/mental-health/fomo/https://socialmediavictims.org/mental-health/fomo/
[Accessed 22 12 2024].

Coursera, 2024. *The History of AI: A Timeline of Artificial Intelligence.* [Online] Available at: https://www.coursera.org/articles/history-of-ai
[Accessed 21 12 2024].

DeAngelis, T., 2020. *Stress in America™ 2020.* [Online] Available at: https://www.apa.org/news/press/releases/stress/2020/report-october?form=MG0AV3
[Accessed 21 12 2024].

Glover, E., 2024. *What Is Artificial Intelligence (AI)?.* [Online] Available at: https://builtin.com/artificial-intelligence#:~:text=Artificial%20intelligence%20(AI)%20is%20a,that%20typically%20require%20human%20intelligence.
[Accessed 21 12 2024].

IBM Data and AI Team, 2023. *Understanding the different types of artificial intelligence.* [Online] Available at: https://www.ibm.com/think/topics/artificial-intelligence-types
[Accessed 21 12 2024].

Kaspersky Lab, 2024. *What are the effects of cyberbullying?.* [Online] Available at: https://www.kaspersky.com/resource-center/preemptive-safety/cyberbullying-effects
[Accessed 22 12 2024].

Mathworks, 2024. *What Is Long Short-Term Memory (LSTM)?.* [Online] Available at: https://www.mathworks.com/discovery/lstm.html
[Accessed 23 12 2024].

Matplotlib, 2025. *Matplotlib: Visualization with Python.* [Online] Available at: https://matplotlib.org/ [Accessed 09 01 2025].

Naslund, J. A., 2012. *Social Media and Mental Health: Benefits, Risks, and Opportunities for Research and Practice.* [Online] Available at: https://pmc.ncbi.nlm.nih.gov/articles/PMC7785056/ [Accessed 22 12 2024].

NumPy , 2024. *NumPy documentation.* [Online] Available at: https://numpy.org/doc/stable/ [Accessed 09 01 2025].

pypi, 2024. *pandas 2.2.3.* [Online] Available at: https://pypi.org/project/pandas/ [Accessed 09 01 2025].

pypi, 2024. *scikit-learn 1.6.0.* [Online] Available at: https://pypi.org/project/scikit-learn/ [Accessed 09 01 2025].

rapidqube, 2024. *gpus role in ai revolution.* [Online] Available at: https://www.rapidqube.com/gpus-role-in-ai-revolution/ [Accessed 21 12 2024].

Roy, R., 2021. *The Naive Bayes classifier.* [Online] Available at: https://towardsdatascience.com/the-naive-bayes-classifier-how-it-works-e229e7970b84 [Accessed 23 12 2024].

RSPH, 2023. *#StatusofMind.* [Online] Available at: https://www.rsph.org.uk/our-work/campaigns/status-of-mind.html [Accessed 21 12 2024].

Saxena, S., 2024. *What is LSTM? Introduction to Long Short-Term Memory.* [Online] Available at: https://www.analyticsvidhya.com/blog/2021/03/introduction-to-long-short-term-memory-lstm/ [Accessed 23 12 2024].

Tensor Flow, 2024. *An end-to-end platform for machine learning.* [Online] Available at: https://www.tensorflow.org/about [Accessed 09 01 2025].

Towards AI, 2021. *Sentiment Analysis with Logistic Regression.* [Online] Available at: https://towardsai.net/p/nlp/sentiment-analysis-with-logistic-regression [Accessed 23 12 2024].

Wankhade, M., 2022. *A survey on sentiment analysis methods, applications, and challenges.* [Online] Available at: https://link.springer.com/article/10.1007/s10462-022-10144-1 [Accessed 23 12 2024].

6 Originality Report

22072054 Anir Jung Thapa

Islington College,Nepal

Document Details

Submission ID
trn:oid:::3618:79796950

Submission Date
Jan 22, 2025, 8:15 AM GMT+5:45

Download Date
Jan 22, 2025, 8:19 AM GMT+5:45

File Name
22072054 Anir Jung Thapa

File Size
57.2 KB

53 Pages

8,099 Words

49,441 Characters

turnitin Page 2 of 59 - Integrity Overview Submission ID trn:oid:::3618:79796950

84% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

- 71 Not Cited or Quoted 84%
  Matches with neither in-text citation nor quotation marks
- 1 Missing Quotations 0%
  Matches that are still very similar to source material
- 0 Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

20% Internet sources
12% Publications
83% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.